



CHECKPOINT 5 · VIBE CODING E MVP · NLP E FRONT-END

OS 7 PECADOS

MVP jogável do RPG da CP4, API FastAPI de IA generativa e diários de desenvolvimento

INTEGRANTES

Guilherme Gama Bitencourt Souza · RM565293
Igor Thiago Nakajima Vieira · RM563632

DATA

9 de outubro de 2026

REPOSITÓRIO

github.com/guilhermegamabs/pln-game-generative-ai

VÍDEOS

PLN: [Grupo: link do vídeo de 2 a 3 min](#) · Front-end: [Grupo: link do vídeo da arquitetura](#)

Sumário

Rascunho: há 8 trecho(s) marcados em amarelo que o grupo ainda precisa preencher (explicação do código, revisões, resultados dos testes manuais). Edite os .md da raiz e gere o PDF de novo.

- 1 O MVP em uma página
- 2 Como executar
- 3 Arquitetura e API (disciplina de Front-end)
- 4 Telas do MVP
- 5 Diário de Vibe Coding
- 6 Diário de Mudanças em relação à CP4
- 7 Checklist de testes

IA **Uso de IA neste trabalho.** O MVP foi desenvolvido com vibe coding (Claude Code), como pede a CP5; o Diário de Vibe Coding (seção 5) registra os prompts reais. A explicação do código na seção 5.4 é texto do grupo. Os resultados de testes e medições apresentados foram executados de fato; os registros brutos estão no repositório.

1 O MVP em uma página

O MVP é a continuação direta da CP4: mesmo título, gênero e premissa. É um RPG narrativo em que o inquisitor chega à Praça do Pelourinho, em Cinzaforte, e decide o destino de Tomás, acusado de roubo, enquanto cada escolha alimenta os sete pecados. A stack é a que a CP4 previu: front-end em React + TypeScript (Vite) e back-end em Python com FastAPI, que expõe o núcleo do jogo já testado na CP4.

Mecânica da CP4	Como funciona no MVP
M1. Diálogo livre	O jogador digita; a LLM classifica a intenção (JSON Schema, temperatura 0) e, numa segunda chamada, o NPC responde em personagem com emoção e memória.
M2. Corrupção	Sete medidores de 0 a 100; a intenção lida e as ações (roubar, doar, atacar, matar) movem os pecados por regras fixas no código.
M3. Demônio Interior	Ao cruzar 40, a manifestação (A Fera, O Mercador) desperta e sussurra com fala gerada pela LLM e voz do Piper; o poder dela pode ser usado em combate, ao custo de mais pecado.
M4. Memória e reputação	NPCs guardam memórias geradas pela LLM; testemunhas lembram de mortes; o segredo só entra no prompt com confiança; pecados altos marcam a aparência.
M5. Combate por turnos	Atacar, defender, usar item (poção, água benta) e poder demoníaco; o NPC revida; vida zerada é derrota.

Requisito da CP5	Onde está
Menu e gameplay do mockup	Menu, Gameplay, Espelho da Alma e Final (seção 4)
Fim de sessão	Veredito sobre Tomás: Redenção, Marcado pela Ira/Avareza; também Consumido (pecado em 100) e Morto em Cinzaforte (derrota em combate)
IA generativa integrada	Texto ao vivo (qwen2.5:7b via Ollama), voz ao vivo (Piper TTS) e imagens do SDXL geradas na CP4, todas pela API do grupo
Extras para a nota 10	2 modalidades em tempo real (texto e voz); 5 mecânicas (mínimo 3); 98 testes automatizados (43 do núcleo, 48 da API, 7 do painel) mais um teste ponta a ponta de 17 passos; fallback para modo offline quando a IA cai; feedback visual (indicador "pensando", animações, barras)

2 Como executar

Requisitos: Python 3.11+, Node 20+, [Ollama](#) com o modelo `qwen2.5:7b` e, para voz, a voz `pt_BR-faber-medium` do Piper. Os READMEs de cada pasta têm os comandos para Windows.

```
ollama pull qwen2.5:7b
python -m venv .venv && .venv/bin/pip install -r api/requirements.txt
cp api/.env.example api/.env          # define API_KEY e PIPER_VOZ (nunca commitar o .env)
cd api && ../.venv/bin/uvicorn main:app --env-file .env      # API em http://localhost:8000 (Swagger em /docs)
cd jogo-web && cp .env.example .env.local                  # VITE_API_KEY = mesma chave
npm install && npm run dev                                # jogo em http://localhost:5173
```

Testes: `prototipo/` (unittest), `api/` e `painel/` (pytest) e `jogo-web/` (`npm run e2e`). Nenhuma chave real está no código: a API lê `API_KEY` do `.env` e recusa tudo se ela não estiver definida.

3 Arquitetura e API (disciplina de Front-end)



Rota	O que faz
POST /v1/ia-generativa/texto	Mensagens + JSON Schema opcional → texto ou JSON gerado pela LLM, modelo e latência
POST /v1/ia-generativa/voz	Texto → áudio WAV do Piper (gestos entre asteriscos não são falados)
GET /v1/ia-generativa/status	Disponibilidade de texto e voz (o jogo decide o modo offline)
POST /v1/jogo/partidas , GET /v1/jogo/partidas/{id} , POST .../comandos	Partidas do jogo web: fala livre ou ação; devolve fala, intenção lida, memória, pecados, combate, manifestação, final e estado

Router e provider: os routers só validam (Pydantic) e traduzem erros em códigos HTTP (401, 404, 409, 422, 502, 503); o provider é a única camada que conhece o Ollama e o Piper. **Autenticação:** header `X-API-Key` , comparado em tempo constante. **CORS:** liberado só para a origem do jogo (Vite, porta 5173). **Swagger:** automático em `/docs` .

Evidência: Swagger, 401 e 200

OS 7 PECADOS · API de IA generativa

1.0.0OAS 3.1

/openapi.json

Back-end da CP5. Encapsula a LLM local (Ollama) e o TTS (Piper) e roda as partidas do jogo web. Rotas em /v1/ia-generativa exigem o header **X-API-Key** (botão *Authorize*).

Authorize

IA generativa

POST/v1/ia-generativa/textoGera texto com a LLM (Ollama)

POST/v1/ia-generativa/vozSintetiza voz com Piper TTS

GET/v1/ia-generativa/statusDisponibilidade de texto e voz

Jogo

POST/v1/jogo/partidasNova partida

GET/v1/jogo/partidas/{id_partida}Estado da partida

POST/v1/jogo/partidas/{id_partida}/comandosFala livre ou ação (/atacar, /roubar, /doar 10, /falar breanna, /veredito absolver...)

infra

GET/healthA API está no ar (sem autenticação)

Schemas

HTTPValidationError > Expand all object

Swagger em /docs: rotas de IA generativa e do jogo, com o botão Authorize.

GET /v1/ia-generativa/status Disponibilidade de texto e voz

O jogo chama ao abrir: se texto=false, entra direto no modo offline em vez de falhar no primeiro turno.

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/v1/ia-generativa/status' \
  -H 'accept: */*'
```

Request URL

http://localhost:8000/v1/ia-generativa/status

Server response

Code Details

401 Error: Unauthorized

Response body

```
{
  "detail": "API Key ausente ou inválida."
}
```

Response headers

```
content-length: 42
content-type: application/json
date: Fri, 09 Oct 2026 20:16:33 GMT
server: uvicorn
vary: Origin
www-authenticate: ApiKey
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "texto": true,
  "voz": true,
  "modelo_texto": "string",
  "detalhe": {
    "additionalProp1": "string",
    "additionalProp2": "string",
    "additionalProp3": "string"
  }
}
```

401 API Key ausente ou inválida

No links

GET /v1/ia-generativa/status sem a chave: 401 “API Key ausente ou inválida”.

GET /v1/ia-generativa/status Disponibilidade de texto e voz

O jogo chama ao abrir: se texto=false, entra direto no modo offline em vez de falhar no primeiro turno.

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/v1/ia-generativa/status' \
  -H 'accept: */*' \
  -H 'X-API-Key: <chave do .env>'
```

Request URL

http://localhost:8000/v1/ia-generativa/status

Server response

Code Details

200

Response body

```
{
  "texto": true,
  "voz": true,
  "modelo_texto": "qwen2.5:7b",
  "detalhe": {
    "texto": "ok",
    "voz": "ok"
  }
}
```

Response headers

```
content-length: 89
content-type: application/json
date: Fri, 09 Oct 2026 20:16:34 GMT
server: uvicorn
vary: Origin
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "texto": true,
  "voz": true,
  "modelo_texto": "string",
  "detalhe": {
    "additionalProp1": "string",
    "additionalProp2": "string",
    "additionalProp3": "string"
  }
}
```

401 API Key ausente ou inválida

No links

Com a chave (Authorize, header X-API-Key): 200, texto e voz disponíveis. A chave foi mascarada no print.

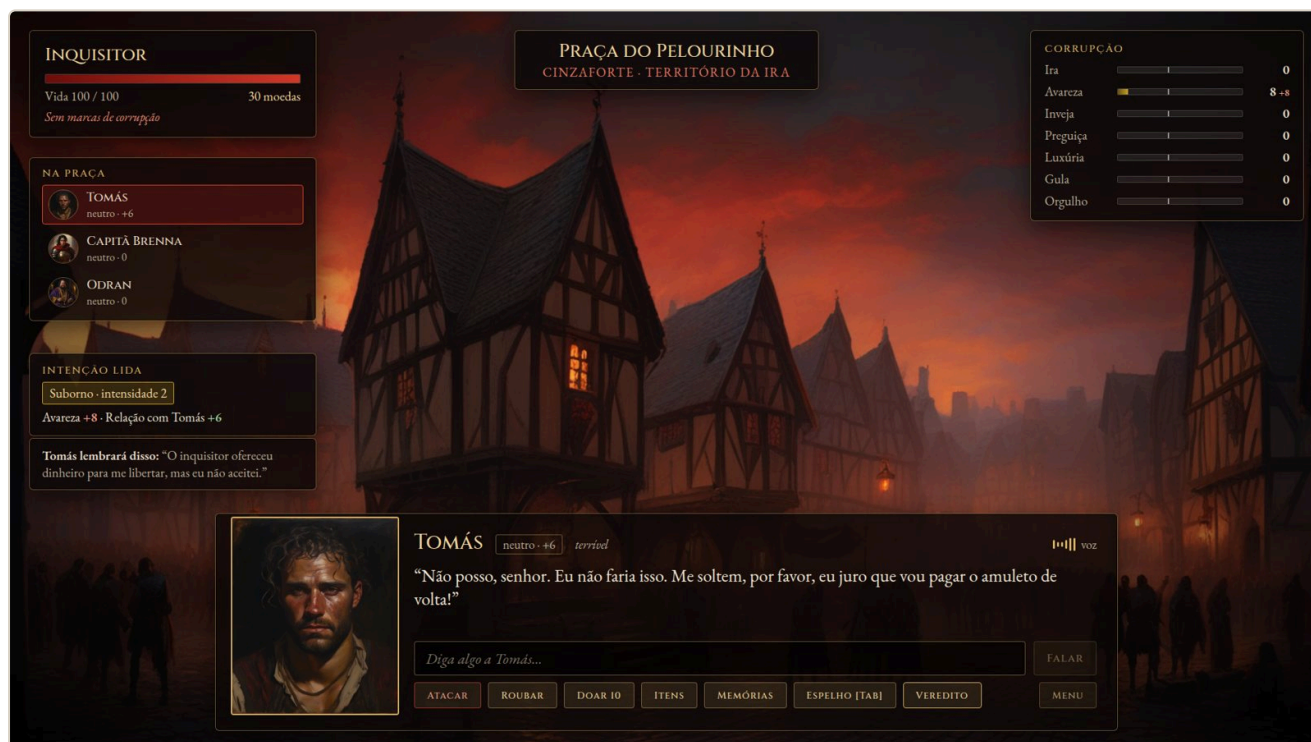
Evidência: o jogo consome a API, não o Ollama

Log do uvicorn durante a coleta das evidências: o 401 e o 200 do Swagger, o jogo de terminal (/status e /texto) e o jogo web (/v1/jogo/partidas e a voz), todos com o qwen2.5:7b real.

```
INFO: 127.0.0.1:43094 - "GET /v1/ia-generativa/status HTTP/1.1" 401 Unauthorized
INFO: 127.0.0.1:43094 - "GET /v1/ia-generativa/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:34776 - "GET /v1/ia-generativa/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:34792 - "POST /v1/ia-generativa/texto HTTP/1.1" 200 OK
INFO: 127.0.0.1:58110 - "POST /v1/ia-generativa/texto HTTP/1.1" 200 OK
INFO: 127.0.0.1:48984 - "POST /v1/ia-generativa/texto HTTP/1.1" 200 OK
INFO: 127.0.0.1:44126 - "GET /v1/ia-generativa/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:44126 - "GET /v1/ia-generativa/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:44120 - "POST /v1/jogo/partidas HTTP/1.1" 201 Created
INFO: 127.0.0.1:44126 - "POST /v1/jogo/partidas/53bd26d264a34d7fb68eb8b1d9c9b1c1/comandos HTTP/1.1" 200 OK
INFO: 127.0.0.1:54978 - "POST /v1/ia-generativa/voz HTTP/1.1" 200 OK
```

Turno real do jogo de terminal, passando pela API:

```
[modelo: qwen2.5:7b via API]
Tomás: Obrigado, senhor... eu juro que não fiz nada com o amuleto. Peço perdão, mas não peguei nada!
[intenção: compaixao (2) · relação com Tomás +16]
```



Jogo web com o modelo real: a fala, a intenção lida e a memória vieram da LLM pela API do grupo.

4 Telas do MVP



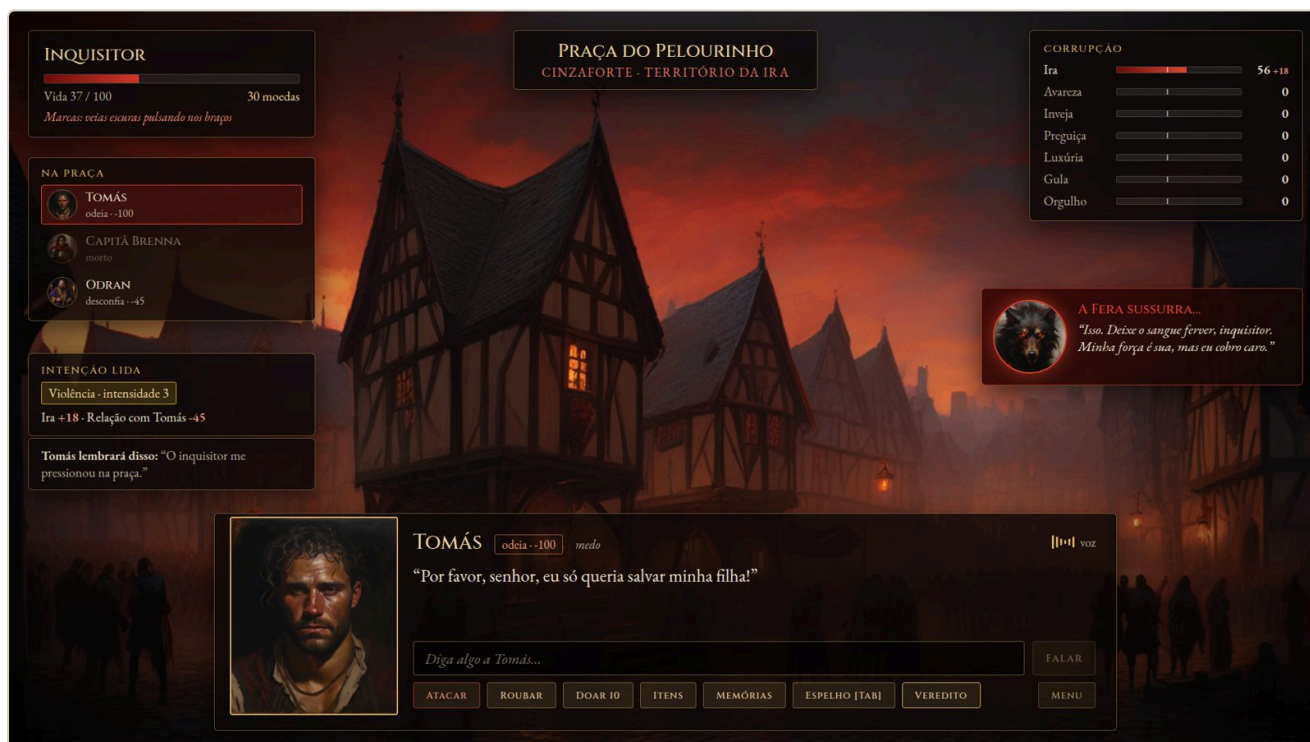
Menu principal: arte SDXL, status do motor de IA e aviso de conteúdo gerado por IA.



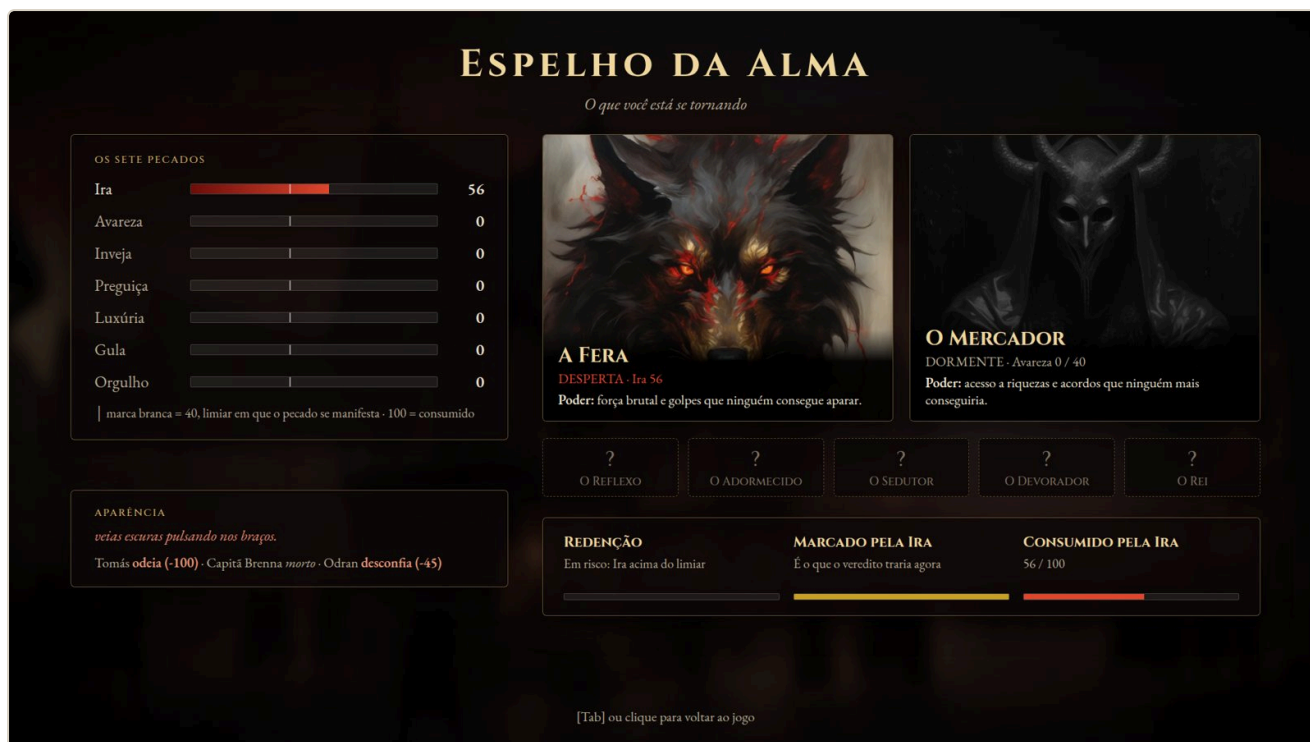
Gameplay: HUD com barra de vida, presentes na praça, medidores, intenção lida, memória e diálogo com retrato e voz.



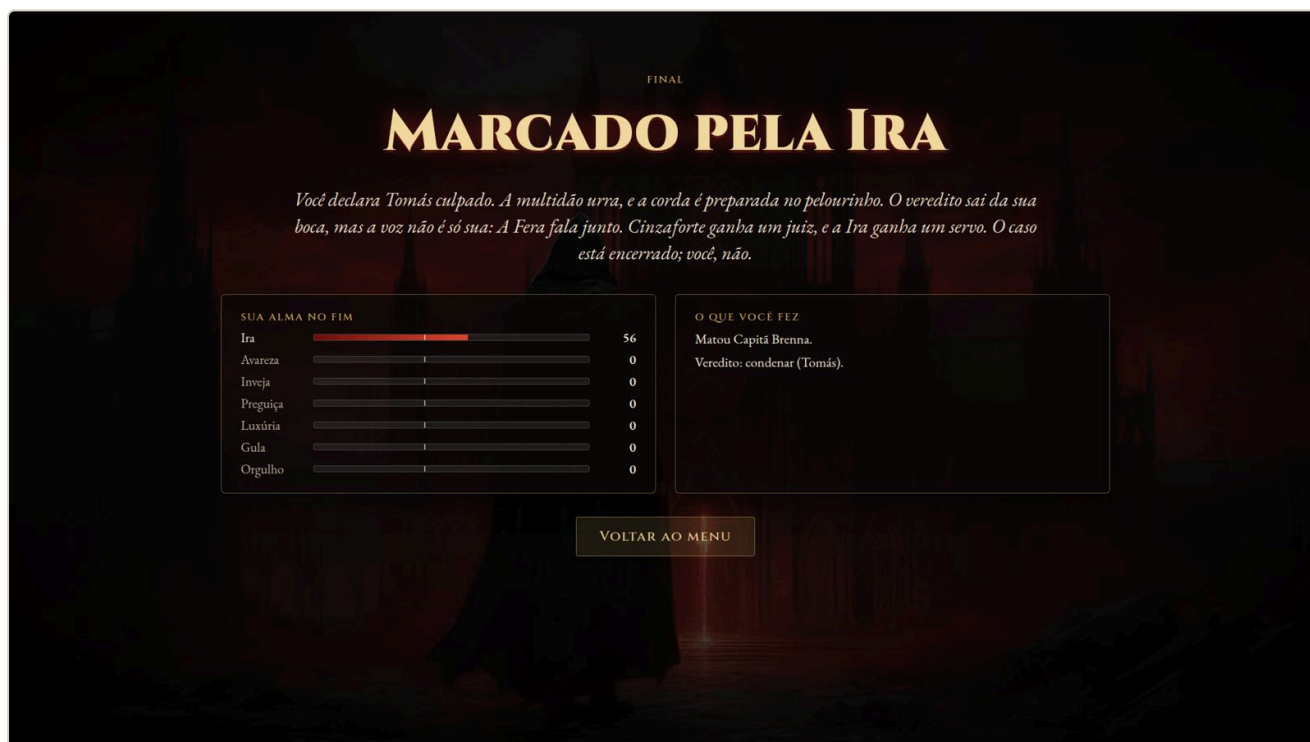
Combate por turnos: vida do oponente ao lado do nome, relato do turno e ações de combate.



Demônio Interior: a Ira passou de 40 e A Fera sussurra com fala gerada pela LLM.



Espelho da Alma (Tab): pecados, aparência, manifestações e progresso para cada final.



Final: o veredito encerra a partida; o desfecho depende dos pecados.

5 Diário de Vibe Coding

Rascunho montado a partir da sessão real de desenvolvimento. Os prompts estão transcritos como foram digitados (com os erros de digitação), e os horários são os da sessão de 09/10/2026. A seção 4, com a explicação do código, **deve ser escrita pelo grupo**, como exige o enunciado.

1. Ferramenta e forma de trabalho

Item	Definição
Ferramenta de vibe coding	Claude Code (CLI no terminal), modelo Claude Opus 5.5
Máquina	Notebook Linux, 14 GB de RAM, sem GPU dedicada
Repositório	Branch <code>feat/cp5-api-fastapi</code> , a partir do commit da CP4 (<code>940d741</code>)
Commits da sessão	<code>273e639</code> (API, jogo pela API, painel), <code>2a31641</code> (testes de integração e correções), <code>b703f27</code> (limpeza de espaços), <code>d71d052</code> (MVP em React)

Como o fluxo funcionou:

1. O grupo escrevia pedidos curtos em português no terminal.
2. A IA lia o repositório da CP4 e o enunciado, propunha um plano e implementava.
3. Em seguida, a IA rodava testes automatizados e partidas reais e relatava o resultado.
4. O grupo decidia o próximo passo, o que entrava no commit e o que não podia ser feito sem confirmação (push, fork, troca de conta, parar processos da máquina).

A IA não tinha permissão para publicar nada no GitHub nem para mexer em processos de outros projetos sem pedido explícito.

2. Linha do tempo resumida

Horário	Prompt do grupo	Resultado
13:47	"na minha pasta documentos, tem um trabalho ai de pln-game-generative-ai"	Leitura do repositório da CP4
13:57	"la em downloads, baixei um pdf tambem"	Leitura do enunciado da CP5 e plano
14:02	"mas o que temos que entregar do front end?"	Lista de requisitos da disciplina de Front-end
14:04	"pode começar pela api"	API FastAPI (prompt-chave 1)
14:09	"pode seguir com o item 1"	Jogo consumindo a API, com fallback (prompt-chave 2)
14:14	"vamos seguir"	Painel Streamlit (prompt-chave 3)
14:17	"pode fazer o commit, mas antes muda pro meu usuario pessoal"	Commit <code>273e639</code>
14:19	"esta tudo certinho? eu quero tirar um 10 nessa materia claude, entao se puder fazer mais testes"	Testes de integração (prompt-chave 4)
14:33 a 14:46	"vou ter que testar aqui", "como posso testar?", "consegue abrir pra mim testar?"	Instalação do Ollama local e comparação de modelos

Horário	Prompt do grupo	Resultado
14:50	"voce nao consegue voce mesmo executar o jogo e fazer os testes?"	Partida roteirizada com o modelo real (prompt-chave 5)
14:58	"pode corrigir os espaços duplos"	Correção <code>b703f27</code> (prompt-chave 6)
15:04	"vamos fazer a parte do guilherme tambem"	MVP em React (prompt-chave 7)
16:54	"quero" (rascunhar os diários)	Rascunho deste diário e do Diário de Mudanças
16:58	"quero que implemente"	Barra de vida, combate por turnos e inventário (prompt-chave 8)
17:07	"o jogo nao ta ocupando a minha tela toda, e parece que esta quebrado"	Correção do palco (prompt-chave 9)
17:12	"pode fazer tudo ai"	Checklist, roteiro dos vídeos, prints de evidência e PDF (prompt-chave 10)

3. Prompts-chave

Prompt-chave 1: a API de IA generativa

Prompt (14:04): "pode começar pela api"

Contexto: logo antes, o grupo tinha perguntado "mas o que temos que entregar do front end?". A IA respondeu com os requisitos do PDF: FastAPI com router e provider, API Key, Swagger, CORS e um painel consumindo a API via requests.

O que a IA gerou:

- A pasta `api/`, com o padrão rota e provider pedido pela disciplina:
 - `routers/ia_generativa.py` com as rotas `POST /v1/ia-generativa/texto`, `POST /v1/ia-generativa/voz` e `GET /v1/ia-generativa/status`;
 - `providers/ia_provider.py`, o único arquivo que chama o Ollama (texto) e o Piper (voz);
 - `seguranca.py`, com a API Key no header `X-API-Key`;
 - `main.py`, com CORS e Swagger em `/docs`.
- 18 testes automatizados, que usam um Ollama simulado porque a máquina ainda não tinha Ollama instalado.

O que foi ajustado, corrigido ou rejeitado:

- O teste "voz sem configuração" passava ou falhava conforme a variável `PIPER_V0Z` do terminal. Foi corrigido para não depender do ambiente.
- Decisão de segurança: se a chave não estiver configurada no servidor, a API recusa tudo (500) em vez de ficar aberta.
- O texto foi testado só com Ollama simulado nesta etapa. O teste com o modelo real ficou registrado como pendência e foi feito no prompt-chave 5.
- *[Grupo: acrescentar aqui o que revisaram ou mudaram nesse código.]*

Prompt-chave 2: o jogo deixa de chamar o Ollama direto

Prompt (14:09): "pode seguir com o item 1"

Contexto: o "item 1" era o requisito da disciplina de Front-end: o jogo precisa consumir a API do grupo, e não mais o serviço de IA diretamente.

O que a IA gerou:

- `ClienteAPI` em `prototipo/llm.py`, com a mesma interface do `ClienteOllama` da CP4. Por isso, o `jogo.py` não precisou mudar.
- Novo padrão no `main.py`: o jogo chama a API. O modo da CP4 continua disponível com `--direto`.
- `ClienteComFallback`: se a API ou o Ollama cair no meio da partida, o jogo avisa uma vez e segue em modo offline.

O que foi ajustado, corrigido ou rejeitado:

- Separação entre dois tipos de erro, para que só a queda do serviço ative o modo offline:
 - **indisponível** (503: conexão recusada, timeout ou modelo não baixado) ativa o fallback;
 - **resposta ruim do modelo** (502) não ativa: o jogo só tenta de novo.
- No `ClienteOllama` antigo, o erro 404 (modelo não baixado) passou a contar como "indisponível".
- *[Grupo: acrescentar.]*

Prompt-chave 3: o painel de consumo via requests

Prompt (14:14): "vamos seguir"

O que a IA gerou:

- `painel/app.py` em Streamlit, com quatro abas: Fala de NPC, Classificar intenção, Voz e Texto livre.
- `painel/cliente_api.py` com `requests`.
- O painel reaproveita os mesmos prompts e schemas do jogo (`prompts.py`), então testa exatamente o que o jogo envia.
- 7 testes com `AppTest`, que clicam nos botões de verdade.

O que foi ajustado, corrigido ou rejeitado:

- O commit só foi feito depois que o grupo pediu para trocar o autor do git para a conta pessoal ("pode fazer o commit, mas antes muda pro meu usuario pessoal"). A troca foi feita apenas neste repositório.
- *[Grupo: acrescentar.]*

Prompt-chave 4: testes de integração para buscar o 10

Prompt (14:19): "esta tudo certinho? eu quero tirar um 10 nessa materia claude, entao se puder fazer mais testes"

O que a IA gerou:

- `api/tests/test_integracao.py`. Os testes sobem o uvicorn numa porta real, com um Ollama simulado que também finge falhas:
 - erro 500;
 - corpo que não é JSON;

- lentidão acima do timeout;
- queda no meio da partida;
- 8 chamadas simultâneas.
- 5 testes com o Piper de verdade.
- Uma partida do jogo inteira passando pela API.

O que os testes acharam e foi corrigido:

1. Um corpo que não era JSON vindo do Ollama derrubava a API com **erro 500 sem tratamento**. Agora devolve 502.
2. Duas chamadas de voz ao mesmo tempo carregavam o modelo do Piper duas vezes. Foi adicionada uma trava (`threading.Lock`).
3. **Não havia limite de tamanho no pedido**. Ficou em 20 mil caracteres por mensagem e 40 mensagens. O valor foi medido nos logs da CP4: o maior prompt real tinha 2,7 mil caracteres.
4. O README mandava rodar `pytest` , mas ele não estava no `requirements.txt` . Isso foi achado clonando o repositório numa pasta limpa e seguindo o README ao pé da letra.
5. **Um teste de concorrência falhava às vezes**. A causa estava no Ollama simulado, não na API: o servidor de teste só aceitava 5 conexões na fila. Corrigido para 64, e o teste passou em 20 execuções seguidas.

Prompt-chave 5: a IA joga a partida com o modelo real

Prompt (14:50): "voce nao consegue voce mesmo executar o jogo e fazer os testes?"

Contexto: o Ollama foi instalado na máquina, e o grupo fechou programas para liberar RAM ("fechei tudo menos o chrome, olha aí"). Antes da partida, a IA mediu os dois modelos da CP4 com o mesmo turno, na CPU:

	qwen2.5:7b	llama3.2:3b
Turnos seguintes	cerca de 1 min	cerca de 30 s
Memória do NPC	gravou memória coerente	voltou vazia nos dois turnos

O 3b foi **rejeitado**: ele quebra a mecânica de memória (M4). O jogo ficou com o qwen2.5:7b, o modelo oficial da CP4.

O que a IA gerou: uma partida roteirizada de 11 passos com o qwen2.5:7b real, passando pela API (log em `prototipo/logs/sessao_20261009_145038_teste-real-via-api.jsonl`):

Passo	Resultado	Tempo
Pergunta ao Tomás	intenção neutra, NPC nervoso	76 s
Suborno ao Odran	suborno, Avareza +8	145 s
Ameaça à Brenna	violência (3), Ira +18	69 s
/atacar	testemunhas registraram a morte	instantâneo
Ameaça ao Tomás	Ira passou de 40 e "A Fera desperta"	122 s
Ollama derrubado	"IA indisponível: HTTP 503", jogo seguiu offline	instantâneo

Problemas encontrados:

- **Espaços duplos em toda a fala da Fera** ("Tu ousou te atrever"). Conferido no log: vinham do próprio modelo. Corrigido no prompt-chave 6.
- Deslizes de português do modelo ("Eu não roubou nada!"). É a qualidade do 7B; não houve correção no código.
- **Latência de 1 a 2,5 min por turno na CPU**. Registrada no Diário de Mudanças.

Prompt-chave 6: limpeza da saída da LLM

Prompt (14:58): "pode corrigir os espaços duplos"

O que a IA gerou:

- A função `_limpar` em `prototipo/jogo.py`, que junta espaços e quebras de linha repetidos. Ela é aplicada à fala do NPC, à memória gravada e à fala da manifestação.
- Um teste que usa a saída real do modelo.

O que foi verificado: o teste falha sem a correção e passa com ela. A memória gravada também ficou limpa, o que evita que o lixo se acumule no histórico enviado nos turnos seguintes.

Prompt-chave 7: o MVP jogável com as telas do mockup

Prompt (15:04): "vamos fazer a parte do guilherme tambem"

O que a IA gerou:

- `jogo-web/` em React, TypeScript e Vite, a stack prevista na seção 4 da CP4. O CSS foi adaptado do `mockup/mockup.html`. Telas:
 - Menu;
 - Gameplay;
 - Espelho da Alma (tecla Tab);
 - Final.
- Rotas de partida na API (`/v1/jogo/partidas`), que expõem o núcleo do protótipo como a CP4 planejou: enviar fala, executar ação e ler estado.
- O comando `/veredito` no núcleo, que fecha a partida com os finais Redenção, Marcado pelo pecado ou Consumido.
- Um teste ponta a ponta (`npm run e2e`): o Chrome sem janela joga 14 passos, do menu ao final.

O que o teste ponta a ponta achou e foi corrigido:

1. O próprio roteiro de teste estava errado. Com uma fala violenta, a Ira ia a 38, abaixo do limiar de 40, e a manifestação não aparecia. Foi acrescentada uma segunda fala violenta.
2. Erro 404 do `favicon` no console do navegador. Corrigido com um ícone vazio.
3. A voz devolvia 503 porque o script de teste não passava `PIPER_V0Z` para a API.
4. A tela Final "pulava" para o lado no fim da animação. O `transform` da centralização brigava com o da animação.
5. As capturas de tela saíam no meio da animação. O teste passou a esperar 1,2 s.

O que foi validado com o modelo real: um turno completo pela interface, em 136 s. Fala, emoção, intenção lida e memória apareceram, sem erros no console.

[Grupo: acrescentar o que revisaram ou mudaram nas telas e nas regras do veredito.]

Prompt-chave 8: combate por turnos, vida e inventário

Prompt (16:58): "quero que implemente"

Contexto: ao rascunhar o Diário de Mudanças, a IA listou as divergências entre a CP4 e o MVP, e as três primeiras eram cortes que ela mesma tinha feito na implementação:

- sem barra de vida (item 1 anotado no mockup da tela de gameplay);
- sem combate por turnos (M5);
- sem inventário.

A IA ofereceu implementar os três para tirar essas linhas do diário, e o grupo aceitou. Foi uma decisão do grupo, contra um corte feito pela IA.

O que a IA gerou:

- No núcleo (`prototipo/jogo.py`), as quatro ações da M5 da CP4:
 - `/atacar` abre o combate e golpeia;
 - `/defender` ;
 - `/item` , com poção e água benta;
 - `/poder` , que só funciona com manifestação desperta e cobra pecado.
- `/fugir` .
- O revide de cada NPC, com vida e dano na ficha (`dados/mundo.json`). Tomás, amarrado, não revida.
- O final de derrota "Morto em Cinzaforte".
- Bloqueio do diálogo e do veredito durante a luta.
- Na tela, a barra de vida no HUD como no mockup, a vida do oponente ao lado do nome e as ações de combate no lugar das ações normais.
- 10 testes novos de combate e itens, mais 3 passos novos no teste ponta a ponta.

O que foi ajustado:

- Para manter o balanceamento validado na CP4, a Ira de matar continuou somando 20, dividida em atacar (+5) e derrubar (+15). Assim os testes antigos de manifestação e de final seguiram valendo, só trocando um ataque por uma luta.
- Os testes antigos que matavam a Brenna com um único `/atacar` passaram a usar uma função que luta até ela cair.
- O teste ponta a ponta acusou "vida 83" onde o roteiro esperava 84. Quem estava errado era o roteiro: ele esquecia que a Brenna, ainda de pé, revida depois do golpe em dobro. O código estava certo, e o roteiro foi corrigido.

Prompt-chave 9: bug de layout achado pelo grupo

Prompt (17:07): "o jogo nao ta ocupando a minha tela toda, e parece que esta quebrado"

Contexto: o grupo jogou no próprio navegador e achou um bug que nenhum teste automatizado tinha pegado.

O que a IA encontrou e gerou:

- O palco de 1920x1080 era centralizado por CSS grid antes da escala. Em janelas menores que 1920 px, ele ficava deslocado para baixo e para a direita, e o diálogo saía cortado. Em janelas fora de 16:9 (o Chrome com barra de abas), sobravam faixas pretas.
- A IA reproduziu o bug com capturas em 5 tamanhos de janela antes de mexer. Depois, a área lógica passou a acompanhar a proporção da janela, e o Espelho e o Final foram para um quadro centralizado.

O que foi ajustado:

- O teste ponta a ponta rodava sempre em exatamente 1920x1080, o único tamanho em que o bug não aparece.
- Ele ganhou um passo que confere o palco em 1536x730, 1280x900 e 2560x1300.

Prompt-chave 10: checklist, evidências e PDF

Prompt (17:12): "pode fazer tudo aí"

O que a IA gerou:

- O checklist de testes manuais (`06_Checklist_Testes.md`).
- O roteiro dos dois vídeos (`07_Roteiro_Videos.md`).
- Os prints do Swagger (401 e 200).
- Um log real da API com o jogo de terminal e o jogo web chamando o qwen2.5:7b.
- O gerador do PDF consolidado (`relatorio/cp5/gerar_pdf.py`), que lê os `.md` e reaproveita o visual do relatório da CP4.

O que foi ajustado, corrigido ou rejeitado:

- **A dificuldade do combate estava errada.** Ao escrever o teste da derrota, a IA percebeu que quase não dava para perder: a Brenna, o NPC mais forte, tirava só 48 de vida numa luta inteira. Ela passou a ter 100 de vida e 30 de dano, e o Odran passou a revidar 10. Agora uma luta descuidada mata, e defender, a poção e o poder do demônio passam a fazer diferença. A rota da derrota foi conferida no código antes de entrar no checklist.
- **O print do Swagger com a chave mostrava a chave real da API**, tanto no campo do Authorize quanto no `curl`. Os prints foram refeitos com a chave mascarada na página. O script de captura falha se a chave ainda aparecer.
- As colunas "Resultado obtido" do checklist ficaram em branco de propósito. O enunciado pede testes realizados pelo grupo.

4. Como o código principal funciona (texto do grupo)

Esta seção precisa ser escrita pelo grupo, com as próprias palavras. O enunciado (Etapa 3) exige uma explicação "em texto próprio do grupo (não gerada por IA)". Os arquivos abaixo são só um guia do que a explicação precisa cobrir; a explicação em si deve vir de vocês.

Arquivos centrais:

- `prototipo/jogo.py`
- `prototipo/prompts.py`
- `prototipo/llm.py`
- `api/routers/`
- `api/providers/ia_provider.py`
- `api/servicos/partidas.py`
- `jogo-web/src/telas/Gameplay.tsx`

[Grupo: escrever aqui.]

6 Diário de Mudanças em relação à CP4

Rascunho baseado no relatório da CP4 (OS_7_PECADOS_CP4.pdf) e no que foi implementado na branch `feat/cp5-api-fastapi`. Os itens marcados com **[Grupo: confirmar]** são decisões de escopo que o grupo precisa validar antes da entrega.

1. O que se manteve igual à CP4

- **Título, gênero e premissa:** OS 7 PECADOS, RPG narrativo de fantasia sombria. O inquisitor corrompido pelos pecados que investiga.
- **Região e cena:** Cinzaforte, na Praça do Pelourinho, com o julgamento de Tomás.
- **Pecados ativos e manifestações:** Ira e Avareza, com A Fera e O Mercador. A CP4 previa "2 ou 3" manifestações.
- **Telas do mockup:** Menu principal, Gameplay e Espelho da Alma. As três foram implementadas a partir do CSS do `mockup/mockup.html`, no mesmo palco de 1920x1080.
- **Mecânicas M1 a M4:**
 - **M1. Diálogo livre:** a intenção é classificada pela LLM em uma chamada e a fala do NPC é gerada em outra.
 - **M2. Sete medidores:** os pontos vêm de regras fixas no código.
 - **M3. Demônio Interior:** desperta quando um pecado chega a 40 e sussurra com uma fala gerada pela LLM.
 - **M4. Memória e reputação:** inclui testemunhas, segredo liberado só com confiança e aparência.
 - **M5. Combate por turnos simples**, com as quatro ações da CP4:
 - atacar;
 - defender (recebe 1/4 do dano e abre uma brecha para o golpe seguinte sair em dobro);
 - usar item (poção de cura e água benta, num inventário);
 - usar o poder demoníaco, que só existe com uma manifestação desperta, causa mais dano e aumenta o pecado dela, como na M3 da CP4.

O revide de cada NPC vem da ficha dele, e os valores de dano ficam fixos no código, como a corrupção.

- **HUD do mockup:** barra de vida, "Vida X / 100", moedas e marcas de corrupção, na mesma posição do mockup da tela de gameplay.
- **Modelos de IA:**
 - qwen2.5:7b via Ollama para texto em tempo real;
 - Piper `pt_BR-faber-medium` para voz;
 - SDXL 1.0 para as imagens já geradas na CP4.

- **Stack:** React, TypeScript e Vite no front e Python com FastAPI no back, como previa a seção 4 da CP4 ("MVP (CP5): back-end" e "MVP (CP5): front-end").
- **Final Consumido:** um pecado chegar a 100 encerra a partida, como na CP4.

2. Tabela de mudanças

Item alterado	O que estava na CP4	O que foi implementado	Justificativa técnica
Número de NPCs	Escopo do MVP: "NPCs: de 5 a 8, todos com memória"	3 NPCs com memória: Tomás, Capitã Brenna e Odran	Cada NPC depende de três coisas: uma ficha validada nos ciclos de prompt v1 a v6 da CP4, um retrato SDXL e a voz. O SDXL precisa da GPU da máquina de produção da CP4 (8 GB de VRAM); o notebook usado na CP5 não tem GPU. Os 3 NPCs da cena já testada cobrem os dois pecados ativos (Brenna é Ira e Odran é Avareza) e o papel de testemunha. Impacto: a cena é curta, mas todas as mecânicas aparecem nela. Novos NPCs entram só como novos registros em <code>prototipo/dados/mundo.json</code> , sem mudar código. <i>[Grupo: confirmar.]</i>
Resolução da região	"Resolver a manifestação local" por combate, negociação ou sacrifício; Redenção avaliada "no confronto final"; combate também contra "criaturas e manifestações"	Nova ação Veredito : o inquisitor absolve ou condena Tomás. Se Tomás estiver morto, o veredito encerra sem decisão. O final é calculado pelos medidores no momento do veredito	O protótipo da CP4 só tinha o final Consumido, mas a CP4 exigia "pelo menos 2 finais" no MVP. Sem um confronto com a manifestação, faltava um momento para avaliar a Redenção. O julgamento de Tomás é o conflito central da cena descrita na CP4, então virou o fim de sessão. O combate por turnos existe, mas só contra os NPCs da praça: lutar contra uma manifestação exigiria uma criatura com ficha e arte próprias, e a manifestação no MVP é uma voz interior, não um corpo na cena. Impacto: é uma mecânica nova, mas usa só regras fixas no código (nenhuma chamada à IA). Está coberta por 4 testes em <code>prototipo/tests/test_jogo.py</code> .
Finais	MVP com pelo menos 2 finais (Redenção vs. Consumido pela Ira/Avareza); na campanha, finais de pecado e "O Oitavo Pecado"	4 finais: Redenção (todos os pecados abaixo de 40), Marcado pela Ira/Avareza (final de pecado, pelo pecado dominante), Consumido (um pecado em 100) e a derrota em combate Morto em Cinzaforte (vida zerada). "O Oitavo Pecado" não foi implementado	Supera o mínimo da CP4. O Oitavo Pecado depende de vários pecados altos ao mesmo tempo, e no MVP só 2 dos 7 estão ativos. Impacto: nenhum negativo.
Latência em tempo real	Medido na CP4: cerca de 5,5 s por turno (qwen2.5:7b, RTX 5060 Ti)	68 a 145 s por turno no notebook de desenvolvimento, que roda tudo em CPU	O notebook usado para desenvolver e testar a CP5 não tem GPU e tem 14 GB de RAM. Foi tentado o llama3.2:3b, a alternativa leve documentada na CP4: ele levou cerca de 30 s por turno, contra cerca de 1 min do 7B, mas devolveu a memória do NPC vazia nos dois turnos testados , o que quebra a M4. Por isso o 7B foi mantido. Impacto: o jogo é o mesmo, mas a fluidez depende do hardware. <i>[Grupo: indicar em qual máquina o vídeo foi gravado. Na máquina com GPU da CP4, a latência volta à medida da CP4.]</i>
Streaming da fala	Mitigação de latência prevista: "streaming da fala (texto aparece enquanto é gerado)"	Indicador "pensando" com contador de segundos. A fala aparece inteira	A fala do NPC vem dentro de um JSON restrito por schema (<code>format</code> do Ollama), junto com emoção e memória. Mostrar o texto aos poucos exigiria um parser de JSON incompleto, além de streaming da API até o navegador. O turno também depende da classificação, que vem antes, e das regras aplicadas depois. Impacto: a espera é visível, mas o jogador vê o tempo passando.

Item alterado	O que estava na CP4	O que foi implementado	Justificativa técnica
Classificador em modelo menor	Mitigação prevista: "classificador no modelo de 3B"	Classificação e fala no qwen2.5:7b	Dois modelos carregados ao mesmo tempo apertam a RAM da máquina de desenvolvimento: com o 7B carregado, restavam cerca de 7,7 GB livres. Revezar os modelos traria de volta a carga de 24 a 67 s medida na CP4. No teste desta máquina, o 3B leu "Solte esse homem ou eu arranco sua cabeça!" como violência, e o 7B como ameaça, o que é mais preciso. Impacto: turnos mais lentos em CPU, mas a classificação fica no modelo validado.
Vozes dos personagens	Vozes pré-geradas por personagem, com efeitos (velocidade, altura, cadeia de efeitos da Fera com voz dupla e reverb); voz feminina para Brenna planejada com Coqui XTTS-v2	A narração de abertura usa o WAV pré-gerado da CP4. As falas geradas ao vivo são sintetizadas em tempo real pela API (<code>POST /v1/ia-generativa/voz</code>), com a voz faber sem efeitos, igual para todos os personagens. Não há voz feminina	As falas do MVP mudam a cada partida, então não podem ser pré-geradas. A cadeia de efeitos da CP4 (<code>geracao/gerar_vozes.py</code>) foi feita para produção offline, e o XTTS-v2 não foi testado no prazo. Impacto: a voz agora acompanha o texto gerado pela LLM (duas modalidades de IA em tempo real), mas com menos caracterização. A Brenna fala com voz masculina.
Contestar a intenção lida	Mitigação prevista: exibir a intenção lida "e permitir contestar ou desfazer"	A intenção lida é exibida (painel "Intenção lida", com intensidade e efeito nos medidores), mas não pode ser contestada	Desfazer exigiria guardar e reverter o estado do turno (pecados, relação, memória e histórico do NPC). Isso ficou fora do prazo. Impacto: a classificação errada continua visível para o jogador, o que já era a primeira parte da mitigação.
Fatos permitidos por NPC e filtro de saída	Planejados contra alucinação e conteúdo impróprio	Não implementados. Continuam os limites no prompt de sistema e a regra do segredo, que só entra no prompt com relação 40 ou mais	Fora do prazo. Impacto: os riscos descritos na CP4 continuam.
Modo offline (fallback)	Não previsto na CP4 para o jogo final (o protótipo tinha <code>--offline</code> só como modo de teste)	Se a API ou o Ollama ficar indisponível (HTTP 503), o jogo avisa e continua com falas simples por palavra-chave. As regras de pecado seguem iguais	É um acréscimo, não uma remoção. Foi testado derrubando o Ollama no meio de uma partida real. Limitação: depois de cair para o offline, a partida segue offline até ser recarregada.
Arquitetura de chamada à IA	Protótipo da CP4 chamava o Ollama direto (<code>prototipo/llm.py</code>)	O jogo chama a API do grupo (FastAPI, <code>X-API-Key</code>), e só a API chama o Ollama e o Piper. O modo antigo continua com <code>--direto</code>	Exigência da disciplina de Front-end da CP5, prevista na seção 4 da CP4 ("Python com API HTTP local (FastAPI)"). Também foi acrescentado um painel Streamlit de testes (<code>painel/</code>). Impacto: mais um processo para subir antes de jogar (API), documentado no README.
Persistência	Save em JSON e log em JSONL, locais ao protótipo	Mesmo formato, mas guardado pela API, um arquivo por partida (<code>api/partidas/{id}.json</code> e <code>.jsonl</code>). O navegador guarda só o id da partida para o "Continuar"	O jogo agora roda no navegador, e o estado e as regras ficam no servidor (decisão da CP4 contra prompt injection). Impacto: a partida sobrevive a reinício da API.

7 Checklist de testes

1. Testes manuais (executados pelo grupo)

As colunas "Resultado obtido" e "OK" ficam para o grupo preencher **depois de executar cada teste**. O enunciado pede testes "realizados pelo grupo"; os resultados esperados abaixo vêm das regras implementadas no código.

Antes de começar:

- 1. Suba o Ollama, a API e o jogo web, como está no `README.md`.
- 2. Abra `http://localhost:5173`.

Para começar cada teste do zero, use **Novo Jogo**.

#	Mecânica ou tela	Passos	Resultado esperado	Resultado obtido	OK
1	Menu principal	Abrir o jogo	Arte do menu (SDXL), opções Novo Jogo, Continuar, Opções, Créditos e Sair; status "Motor narrativo: qwen2.5:7b via API do grupo · pronto"		
2	Narração (voz pré-gerada)	Clicar na página e depois em Novo Jogo	Narração de abertura em voz Piper; gameplay diante de Tomás, Vida 100 / 100, 30 moedas		
3	M1. Diálogo livre	Digitar "Calma, Tomás. Eu vou te ajudar."	Indicador "pensando" com segundos; fala do Tomás em personagem, com emoção e voz		
4	M1. Intenção lida	Ver o painel "Intenção lida" após o teste 3	Compaixão, intensidade 1 a 3, com o efeito na relação		
5	M4. Memória	Ver o aviso "lembrará disso" e o botão Memórias	A memória gerada pela LLM aparece e fica na lista de Memórias do Tomás		
6	M2. Corrupção por fala	Falar com Odran: "Te pago 20 moedas para retirar a acusação"	Intenção Suborno; a Avareza sobe no medidor		
7	M2. Ação Roubar	Com Odran, clicar em Roubar	+25 moedas; Avareza +15; Odran lembra do roubo		
8	M2. Ação Doar	Clicar em Doar 10	-10 moedas; Avareza cai; relação +10		
9	M5. Início do combate	Com Brenna, clicar em Atacar e confirmar	Pede confirmação; abre o combate; Brenna 75 / 100; você 70 / 100; Ira +5; diálogo travado		
10	M5. Defender	No combate, clicar em Defender	Você recebe só 8 de dano (1/4 de 30); o relato fala em "Brecha aberta"		
11	M5. Golpe na brecha	Clicar em Golpear logo depois	50 de dano (golpe em dobro); Brenna revida 30		
12	M5. Item em combate	Clicar em Poção	+35 de vida (até 100); o contador de poções cai; Brenna revida		
13	M5 e M4. Derrubar e testemunhas	Golpear até a Brenna cair	Brenna "morto" na praça; Ira +15; Tomás e Odran "viram tudo"; o combate termina		

#	Mecânica ou tela	Passos	Resultado esperado	Resultado obtido	OK
14	M3. Demônio Interior	Com Tomás, digitar "Eu vou te matar, ladrão!" até a Ira passar de 40	"A Fera sussurra" com retrato e fala gerada pela LLM e com voz; marca "veias escuras" no HUD		
15	M5 e M3. Poder demoníaco	Com a Fera desperta, atacar Odran e clicar em "Poder: A Fera"	45 de dano (Odran cai); Ira +8		
16	M5. Fugir	Num combate, clicar em Fugir	O combate termina; o NPC continua ferido e lembra da fuga		
17	Espelho da Alma	Apertar Tab	7 medidores, aparência, relações, cartas da Fera e do Mercador, 5 manifestações bloqueadas e 3 finais com progresso		
18	Final Redenção	Novo Jogo, uma fala gentil, Veredito e Absolver	Tela Final "Redenção"		
19	Final Marcado	Com a Ira acima de 40, Veredito e Condenar	Tela Final "Marcado pela Ira"		
20	Final Consumido	Levar a Ira a 100 (atacar e ameaçar várias vezes)	"Consumido pela Ira"		
21	Final Derrota	Novo Jogo; atacar Odran até ele cair; depois atacar a Brenna usando só Golpear	Odran tira 10; Brenna tira 30 por turno; no 3º reveide dela a vida chega a 0: tela "Morto em Cinzaforte"		
22	Continuar	Fechar a aba no meio da partida e abrir de novo	Continuar retoma com os mesmos pecados, a mesma vida e o mesmo combate		
23	Fallback offline	Durante a partida, parar o Ollama e falar com um NPC	Aviso "IA indisponível: modo offline"; o jogo segue com falas simples, sem travar		
24	Tamanho da janela	Redimensionar a janela do navegador	O jogo ocupa a janela inteira, sem faixas pretas nem corte		
25	API: autenticação	No Swagger (/docs), executar GET /v1/ia-generativa/status sem Authorize	401 "API Key inválida ou ausente"		
26	API: com chave	Authorize com a chave e executar de novo	200 com texto: true e voz: true		
27	Evidência	Olhar o terminal da API durante uma fala no jogo	Linhas POST /v1/jogo/partidas/.../comandos 200 ; o jogo nunca chama a porta 11434		
28	Painel Streamlit	streamlit run painel/app.py , aba "Fala de NPC", Enviar	Intenção, fala, memória e áudio tocando		

Balanceamento: o inquisitor tem 100 de vida. Brenna tem 100 de vida e revida 30; uma luta só com golpes custa 90 de vida. Odran tem 40 de vida e revida 10. Tomás, amarrado, não revida. Vencer a Brenna com folga exige defender, usar poção ou aceitar o poder do demônio, que cobra pecado.

2. Testes automatizados

Rodados antes da entrega. Comandos no README.md de cada pasta.

Suíte	Onde	O que cobre	Resultado
Núcleo do jogo	<code>prototipo/tests</code> (unittest, 43 testes)	Classificação antes da fala, segredo só com confiança, memória sem repetir, corrupção, manifestação ao cruzar 40, Consumido, veredito e finais, combate (revide, defesa, poder, fuga, derrota, itens, save no meio da luta), limpeza de espaços da LLM	43 OK
API	<code>api/tests</code> (pytest, 48 testes)	Autenticação 401, validação 422, erros 502 e 503, CORS, Swagger exigindo chave, servidor real com Ollama simulado (falhas, lentidão, 8 chamadas simultâneas), Piper real (WAV válido, gestos não falados, velocidade), partidas do jogo pela API	48 OK
Painel	<code>painel/tests</code> (Streamlit AppTest, 7 testes)	Botões clicados de verdade; avisos de 401 e 503	7 OK
Ponta a ponta	<code>jogo-web</code> (<code>npm run e2e</code> , 17 passos)	Chrome sem janela joga do menu ao final: diálogo, combate completo, Fera, Espelho, Memórias, veredito, Continuar, tamanhos de janela; confere que o navegador só chama a API do grupo e que não há erro no console	17 OK
Com o modelo real	Partida roteirizada pelo terminal e turnos pelo jogo web, com qwen2.5:7b via API	Suborno, violência, manifestação, testemunhas, memória e fallback ao derrubar o Ollama	Detalhes no Diário de Vibe Coding, prompt-chave 5